# SMART MODULATION ADVISOR

Revised Blueprint v2: a build-ready plan combining the interface design with a realistic, Python-based implementation

## One-line idea

An interactive app where the user sets the signal quality (SNR) and a maximum acceptable error rate (BER). The app calculates the BER of BPSK, QPSK and 16-QAM and recommends the **fastest scheme that still meets the BER limit**.

**Central rule:** the calculated BER, not fixed visual thresholds, drives every recommendation.

| Item | Decision |
|---|---|
| Language and app framework | Python + Streamlit (one codebase, fast to build) |
| Modulations | BPSK, QPSK, 16-QAM (Gray mapped) |
| Channel | AWGN (additive white Gaussian noise) |
| SNR definition | Es/N0 (energy per symbol over noise density), in dB |
| Default BER limit | 1e-3 (adjustable) |
| Repository | smart-modulation-advisor on GitHub (two collaborators) |

## 1. Are the two plans the same?

**Short answer:** the core idea is identical, but the earlier interface blueprint is much larger than what can be finished and explained well in one night. This revised version keeps the same idea and the same rule, and fixes the points below.

| Topic | Original interface blueprint | Revised v2 (this document) |
|---|---|---|
| Core idea | Recommend fastest modulation meeting a BER limit | Same |
| Technology | HTML, CSS, JavaScript, Chart.js (Python optional) | Python + Streamlit. The simulator is already in Python, so no rewrite in JavaScript is needed. |
| Size | 7 pages plus 3 extra mockups | 1 app with 4 tabs; the rest listed as future scope |
| SNR definition | Not specified | Es/N0 is stated clearly. With Eb/N0, BPSK and QPSK have the same BER and the advisor could never prefer BPSK. |
| BER method | Theoretical, Monte Carlo or both (optional) | Both, with a toggle. The Monte Carlo curve is checked against theory, which is the credibility step. |
| Edge cases | Not covered | A clear message when no scheme meets the limit; reproducible random seed |
| Testing | Not covered | pytest tests for BER trend, ordering and recommendation |
| GitHub and teamwork | Not covered | Repo layout, branches, pull requests, commit habits |
| Timeline | Three phases, no times | Overnight Phase 1 plan with a priority order |
| Skills and resume | Not covered | Skills list and resume bullets (only for finished work) |

**What is kept from the original blueprint:** dark navy and blue/purple look, SNR slider, BER vs SNR graph with the limit line, recommendation card with calculated BER, bits per symbol, switching-point display, constellation diagrams, the honesty notes (labelled theory vs simulation, no claim of controlling real WiFi), and the central rule.

## 2. Technical decisions that make the advisor correct

**2.1 SNR as Es/N0.** The simulator sets average symbol energy to 1. Noise standard deviation per dimension is sqrt(1 / (2 x Es/N0)). Es/N0 in dB converts with 10^(dB/10).

**2.2 Constellations as lists of points.** Each scheme is a list of allowed points (2, 4, 16), scaled so average energy is 1 (divide QPSK by sqrt(2), 16-QAM by sqrt(10)). The receiver picks the nearest point. One code path then works for all three schemes.

**2.3 Theoretical BER (AWGN, Gray mapping, per bit):**

| Scheme | Bits per symbol | Theoretical BER in terms of Es/N0 (x = Es/N0, linear) |
|---|---|---|
| BPSK | 1 | Q( sqrt(2x) ) |
| QPSK | 2 | Q( sqrt(x) ) |
| 16-QAM | 4 | approx (3/4) x Q( sqrt(x / 5) ), a high-SNR approximation |

Q(z) is the Gaussian tail probability, computed as 0.5 x erfc(z / sqrt(2)) using SciPy. The 16-QAM formula is an approximation, so label it as one in the app.

**2.4 Recommendation rule.** Keep every scheme whose BER is at or below the limit, then choose the one with the most bits per symbol. If none qualify, show 'No scheme meets the limit at this SNR'.

**2.5 Sanity check.** For a 1e-3 limit, the theory above puts the switching points near 7 dB (BPSK becomes usable), 10 dB (QPSK) and 16 to 17 dB (16-QAM). These are expected values for checking your results, not fixed thresholds. The app must compute them, never hard-code them.

**2.6 Monte Carlo accuracy.** To measure a BER near 1e-3 reliably you need at least about 100,000 bits (about 100 errors). For smaller BER values, use the theoretical curve or more bits.

**2.7 Reproducibility.** Use a seeded random generator so results do not jump on every slider move.

## 3. Features by phase

| Phase | Features | Priority |
|---|---|---|
| Phase 1 (tonight) | Simulator core for 3 schemes; BER vs SNR plot (log scale) with BER limit line; SNR slider; BER limit input; recommendation card (scheme, calculated BER, bits per symbol, one-line reason); no-scheme-qualifies message | Must have |
| Phase 2 (next day) | Theory vs Monte Carlo toggle; constellation diagrams with noise scatter; switching-point table (SNR where each scheme becomes eligible); bits-per-symbol comparison; pytest tests; README with screenshots | Should have |
| Phase 3 (later) | Adaptive switching step graph; signal-quality labels (marked illustrative); downloadable results (CSV); error-correcting code option; fading channel; more schemes (8-PSK, 64-QAM); live deployment link | Nice to have |

**Interface plan (Streamlit, dark navy theme):** one app with a sidebar for controls and four tabs.

| Tab | Content | Replaces page in original blueprint |
|---|---|---|
| Dashboard | SNR slider, BER limit, recommendation card, BER graph | Home + Simulator |
| Performance | BER comparison, performance table, switching points | Performance Analysis + Insights |
| Theory | BPSK/QPSK/16-QAM explanations, constellation diagrams | Modulation Theory |
| About | Problem, method, assumptions, limitations, future scope, team | About + Settings notes |

Settings (BER limit, SNR range, number of bits) live in the sidebar instead of a separate page. Streamlit's theme can be set in *.streamlit/config.toml* to match the navy and blue style.

# 4. GitHub repository

```
smart-modulation-advisor/
  simulator/
    __init__.py
    simulator.py        constellations, noise, BER, best_method
    theory.py           theoretical BER formulas
  app/
    streamlit_app.py    sliders, plots, recommendation card
  tests/
    test_simulator.py
  docs/
    Smart_Modulation_Advisor_Interface_Blueprint.pdf
    Smart_Modulation_Advisor_Revised_Blueprint_v2.pdf
  results/              saved plots (ber_curves.png, screenshots)
  .streamlit/config.toml
  requirements.txt
  README.md
  LICENSE
```

## 4.1 Setup commands

```
git clone https://github.com/<username>/smart-modulation-advisor.git
cd smart-modulation-advisor
python -m venv venv
venv\Scripts\activate          (Windows)   |   source venv/bin/activate  (Mac/Linux)
pip install numpy matplotlib scipy streamlit pytest
pip freeze > requirements.txt
```

## 4.2 Teamwork workflow

- Create the repo as public with a README, a Python .gitignore and an MIT licence; add the teammate under Settings, Collaborators.
- Work on branches (for example *feature/simulator* and *feature/app*); never both edit the same file on main.
- Open a pull request for each finished piece and let the other person review and merge it.
- Run *git checkout main* and *git pull* before starting new work.
- Commit small and often with clear messages, such as 'Add 16-QAM constellation' or 'Add BER limit slider'.
- Aim for 10 to 15 meaningful commits and at least two pull requests before sharing the link.

## 4.3 README checklist

- One-line description and a screenshot or short GIF at the top
- How to run: pip install -r requirements.txt, then streamlit run app/streamlit_app.py
- Short theory section: the three schemes, SNR (Es/N0), BER and the recommendation rule
- Results: BER curves and the computed switching points
- Validation: simulated vs theoretical curves
- Limitations: simulation only, AWGN channel, no coding, does not control real WiFi/4G/5G
- Future scope and who built what

# 5. Overnight build order

| Step | Task | Done when |
|---|---|---|
| 1 | Create repo, clone, set up environment, first push | Files visible on GitHub |
| 2 | Paste and run simulator; print best scheme at 4, 10, 20 dB | Output goes BPSK, QPSK, 16-QAM |
| 3 | Plot BER vs SNR for all three (log scale) and compare with theory | Simulated and theory curves overlap |
| 4 | Build Streamlit page: slider, BER limit, graph, recommendation card | App runs locally |
| 5 | Add 3 to 4 pytest tests | pytest passes |

| Step | Task | Done when |
|---|---|---|
| 6 | README with screenshot; push; open a pull request | Repo looks complete |

**If time runs short,** finish in this order: simulator, BER plot, advisor, app, tests, README. The first three alone already form a good project.

**Suggested tests:** BER decreases as SNR increases for every scheme; at 30 dB, 16-QAM is recommended; at a very low SNR, no scheme or only BPSK qualifies; at the same SNR, BER(BPSK) <= BER(QPSK) <= BER(16-QAM).

## 6. Technical skills gained

List these on a resume only after the repository is working and you can explain each one.

| Area | Skills |
|---|---|
| Communications (ECE) | BPSK, QPSK, 16-QAM; constellations and Gray mapping; AWGN channel; SNR (Es/N0) and BER; theoretical vs simulated analysis; adaptive modulation; speed vs reliability trade-off |
| Programming | Python, NumPy, SciPy, Matplotlib; vectorised simulation; Monte Carlo methods; modular code design |
| Software engineering (CSE) | Streamlit app development; unit testing with pytest; Git, GitHub branches and pull requests; README and documentation; dependency management |
| Analysis | Validating a simulation against theory; choosing sample sizes for reliable BER estimates; decision logic driven by computed data |

## 7. Resume bullets (use once built)

**Smart Modulation Advisor** | Python, NumPy, SciPy, Streamlit, pytest | GitHub link

- Built a Python simulator of BPSK, QPSK and 16-QAM over an AWGN channel and validated Monte Carlo BER curves against theoretical results.
- Implemented an adaptive modulation advisor that recommends the highest-rate scheme meeting a user-set BER limit, with computed switching points.
- Developed an interactive Streamlit dashboard with live BER plots and constellation diagrams; added unit tests and collaborated through GitHub pull requests.

Until the project is finished, label it '(in progress)' and describe only what works. Do not add percentages or results that you have not measured.

## 8. Assumptions and honest limitations

- This is a simulation of the principle of adaptive modulation. It does not control a real WiFi, 4G or 5G connection.
- The channel is AWGN only: no fading, interference, coding or hardware effects.
- Es/N0 is used throughout. Results in Eb/N0 would look different, so always state which one is shown.
- 16-QAM theory is an approximation, labelled as such. Monte Carlo results are limited by the number of bits.
- Signal-quality labels (Poor, Moderate, Good, Excellent) are illustrative, not standards.
- Switching points depend on the chosen BER limit and SNR definition; they are computed, never fixed.

## 9. Future scope

- Error-correcting codes to lower BER at the same SNR
- Rayleigh fading channel for mobile conditions
- More schemes (8-PSK, 64-QAM) and a throughput estimate
- Live deployment on Streamlit Community Cloud and downloadable CSV results